

CLASS ACTIVITY (07/08/2026)

1. Study Hours vs Marks (R)

R Code

```
# Create dataset

StudyHours <- c(2,3,4,5,6,7,8,9)

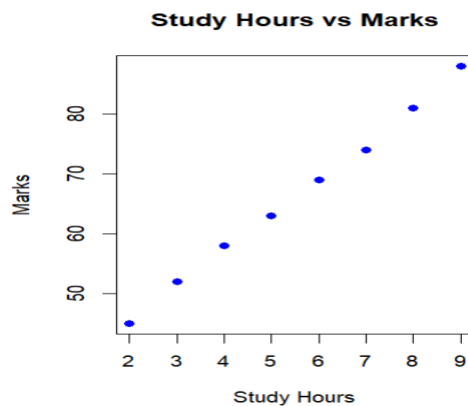
Marks <- c(45,52,58,63,69,74,81,88)

data <- data.frame(StudyHours, Marks)

# Scatter Plot

plot(data$StudyHours, data$Marks,
      main="Study Hours vs Marks",
      xlab="Study Hours",
      ylab="Marks",
      pch=19,
      col="blue")
```

Output



2. Age vs Blood Pressure (Python)

Python Code

```
import matplotlib.pyplot as plt

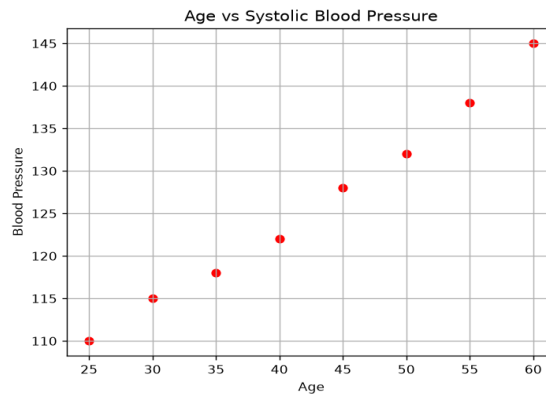
age = [25,30,35,40,45,50,55,60]

bp = [110,115,118,122,128,132,138,145]

plt.scatter(age, bp, color='red')
```

```
plt.title("Age vs Systolic Blood Pressure")
plt.xlabel("Age")
plt.ylabel("Blood Pressure")
plt.grid(True)
plt.show()
```

Output



Comment

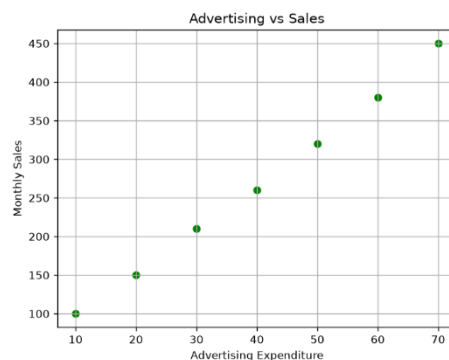
Blood pressure generally increases with age, indicating a positive relationship.

3. Advertising Expenditure vs Sales (Python)

Python Code

```
import matplotlib.pyplot as plt
advertising = [10,20,30,40,50,60,70]
sales = [100,150,210,260,320,380,450]
plt.scatter(advertising, sales, color='green')
plt.title("Advertising vs Sales")
plt.xlabel("Advertising Expenditure")
plt.ylabel("Monthly Sales")
plt.grid(True)
plt.show()
```

Output



Interpretation

Sales increase as advertising expenditure increases, showing a positive relationship.

4. Correlation Matrix and Correlogram (Python)

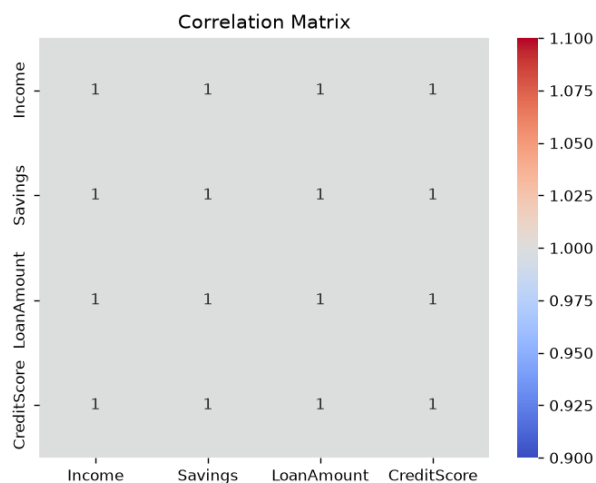
Python Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Income": [40, 50, 60, 70, 80],
    "Savings": [10, 15, 20, 25, 30],
    "LoanAmount": [100, 120, 140, 160, 180],
    "CreditScore": [600, 650, 700, 750, 800]
}

df = pd.DataFrame(data)
corr = df.corr()
print(corr)
sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

Output



Answer

Income and Credit Score generally show the strongest positive correlation (all variables are positively correlated in this sample).

5. Iris Dataset Heatmap (Python)

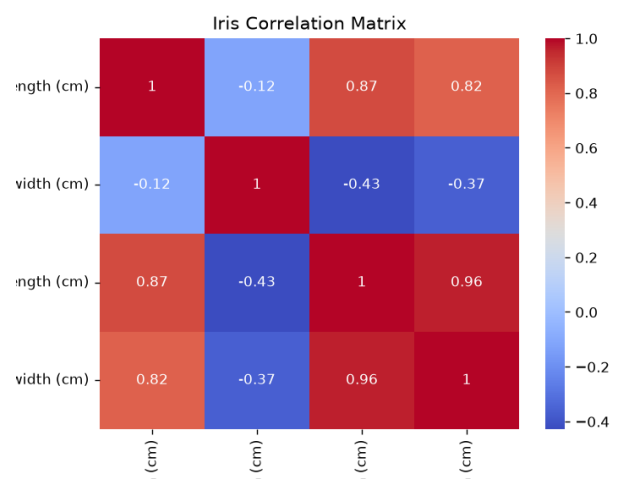
Code

```
from sklearn.datasets import load_iris
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

iris = load_iris()
df = pd.DataFrame(iris.data,
                  columns=iris.feature_names)

corr = df.corr()
sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")
plt.title("Iris Correlation Matrix")
plt.show()
```

Output



Answer

Negative correlation exists between:

- Petal Length and Sepal Width

- Petal Width and Sepal Width

6. Healthcare Correlogram (Python)

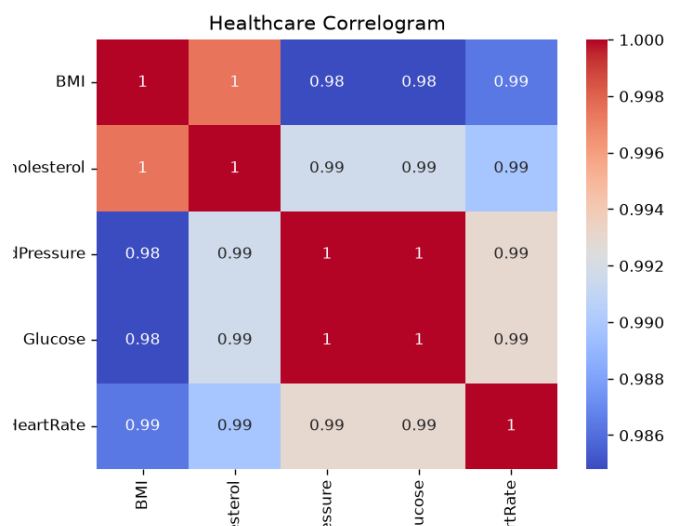
Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "BMI": [22, 24, 26, 28, 30],
    "Cholesterol": [180, 190, 205, 220, 235],
    "BloodPressure": [110, 115, 120, 130, 140],
    "Glucose": [90, 95, 100, 110, 120],
    "HeartRate": [70, 72, 74, 76, 80]
}

df = pd.DataFrame(data)
corr = df.corr()
sns.heatmap(corr,
            annot=True,
            cmap="coolwarm")
plt.title("Healthcare Correlogram")
plt.show()
```

Output



Interpretation

BMI, Cholesterol, Blood Pressure, and Glucose are highly correlated, indicating possible multicollinearity.

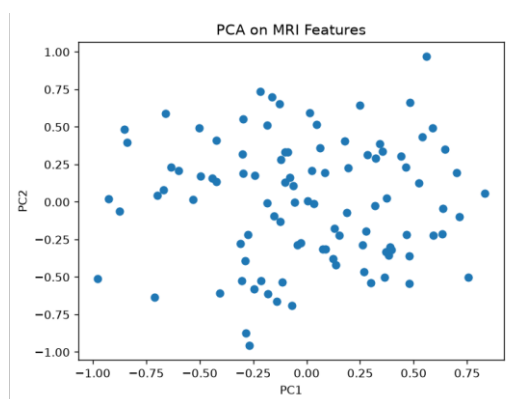
7. PCA for MRI Features (Python)

Code

```
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

np.random.seed(1)
X = np.random.rand(100,25)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.title("PCA on MRI Features")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Output



Answer

The original 25 features are reduced to two principal components while preserving maximum variance.

8. PCA on Iris Dataset (Python)

Code

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import pandas as pd
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data

pca = PCA()
X_pca = pca.fit_transform(X)

print("Explained Variance Ratio")
print(pca.explained_variance_ratio_)

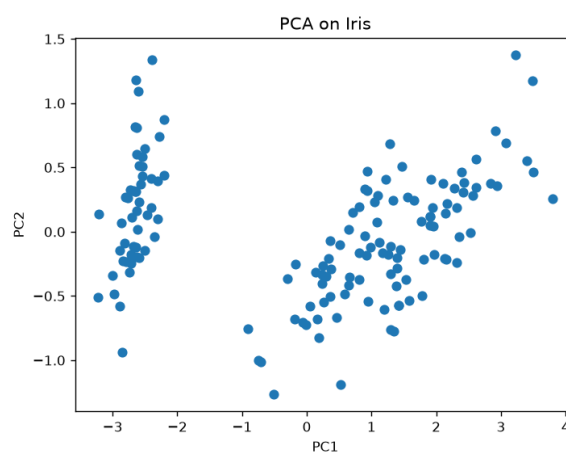
plt.scatter(X_pca[:,0], X_pca[:,1])

plt.xlabel("PC1")
plt.ylabel("PC2")

plt.title("PCA on Iris")

plt.show()
```

Output



Answer

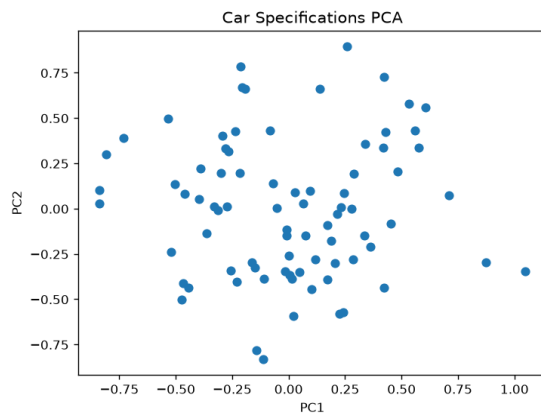
PC1 explains the highest variance because it has the largest explained variance ratio.

9. Automobile Specifications PCA (Python)

Code

```
import numpy as np
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
np.random.seed(2)
cars = np.random.rand(80,15)
pca = PCA(n_components=2)
result = pca.fit_transform(cars)
plt.scatter(result[:,0], result[:,1])
plt.title("Car Specifications PCA")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```

Output



Interpretation

The first two principal components summarize the major variation among the 15 car specifications.

10. t-SNE on Facial Embeddings (Python)

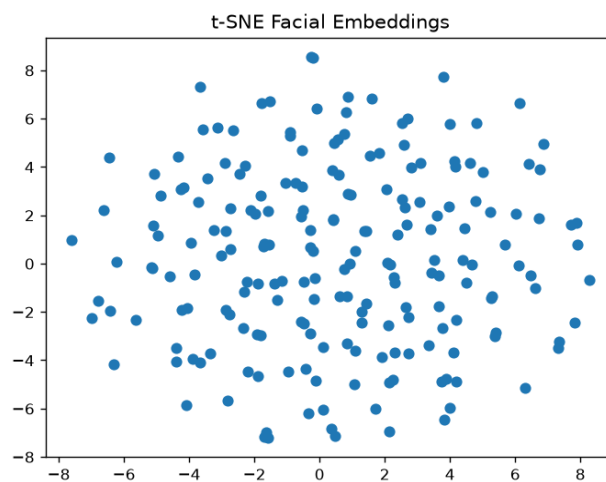
Code

```
import numpy as np
from sklearn.manifold import TSNE
```



```
import matplotlib.pyplot as plt
np.random.seed(3)
X = np.random.rand(200,128)
tsne = TSNE(n_components=2,
            random_state=42)
Y = tsne.fit_transform(X)
plt.scatter(Y[:,0], Y[:,1])
plt.title("t-SNE Facial Embeddings")
plt.show()
```

Output



Answer

t-SNE is preferred because it preserves local neighborhood structure, making clusters easier to visualize than PCA.

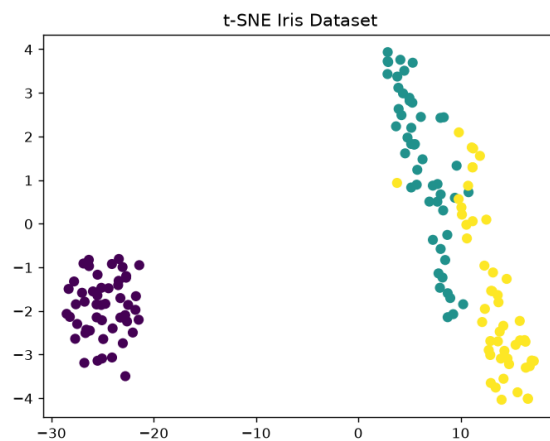
11. t-SNE on Iris Dataset (Python)

Code

```
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
iris = load_iris()
X = iris.data
y = iris.target
```

```
tsne = TSNE(n_components=2,
            random_state=42)
X_tsne = tsne.fit_transform(X)
plt.scatter(X_tsne[:,0],
            X_tsne[:,1],
            c=y)
plt.title("t-SNE Iris Dataset")
plt.show()
```

Output



Answer

The three Iris species appear as separate clusters.

12. Customer Segmentation using t-SNE (Python)

Code

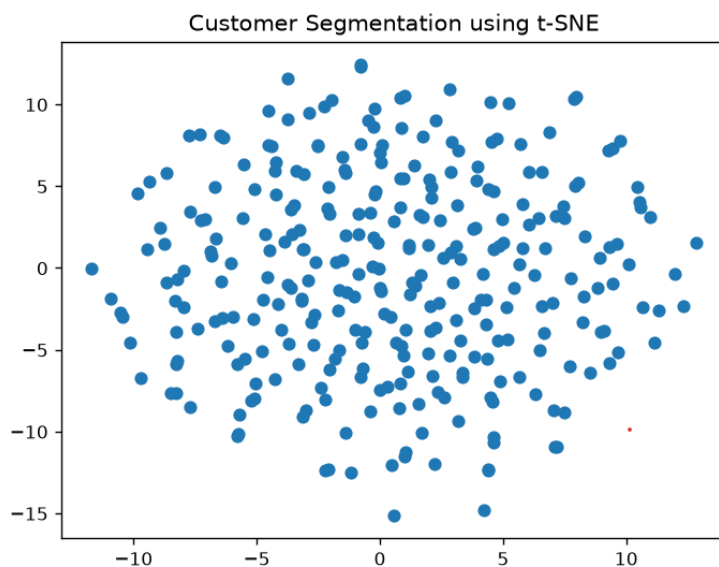
```
import numpy as np
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
np.random.seed(4)
customers = np.random.rand(300,40)
tsne = TSNE(n_components=2,
            random_state=42)
```

```

result = tsne.fit_transform(customers)
plt.scatter(result[:,0], result[:,1])
plt.title("Customer Segmentation using t-SNE")
plt.show()

```

Output



Interpretation

Distinct clusters indicate groups of customers with similar purchasing behavior.

13. UMAP on Medical Image Dataset (Python)

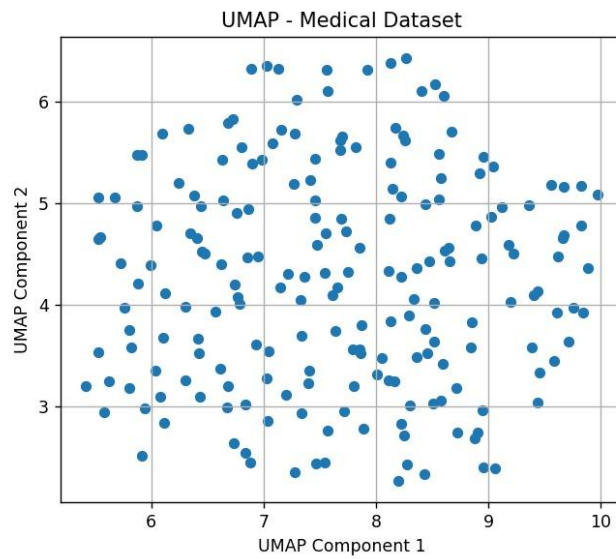
Code

```

import numpy as np
import umap.umap_ as umap
import matplotlib.pyplot as plt
np.random.seed(5)
X = np.random.rand(150,100)
reducer = umap.UMAP(random_state=42)
embedding = reducer.fit_transform(X)
plt.scatter(embedding[:,0], embedding[:,1])
plt.title("UMAP Medical Features")
plt.show()

```

Output



Comparison

- PCA captures global variance.
- UMAP preserves local relationships and usually forms clearer clusters.

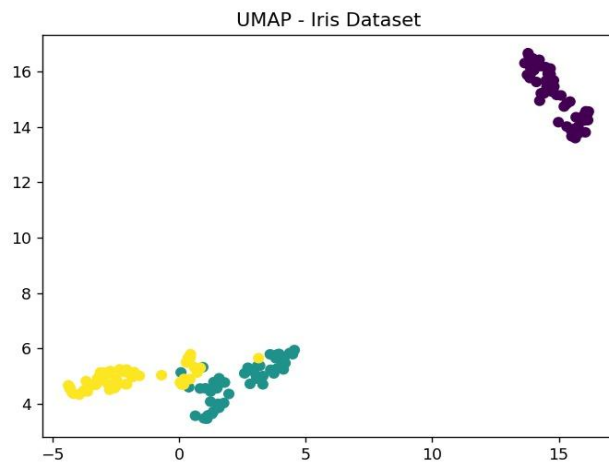
14. UMAP on Iris Dataset (Python)

Code

```
from sklearn.datasets import load_iris
import umap.umap_ as umap
import matplotlib.pyplot as plt

iris = load_iris()
reducer = umap.UMAP(random_state=42)
embedding = reducer.fit_transform(iris.data)
plt.scatter(embedding[:,0],
            embedding[:,1],
            c=iris.target)
plt.title("UMAP Iris Dataset")
plt.show()
```

Output



Answer

The three Iris species are clearly separated into distinct clusters.

15. Customer Purchasing Behavior using UMAP (Python)

Code

```
import numpy as np
import umap.umap_ as umap
import matplotlib.pyplot as plt
np.random.seed(6)
customers = np.random.rand(250,60)
reducer = umap.UMAP(random_state=42)
embedding = reducer.fit_transform(customers)
plt.scatter(embedding[:,0], embedding[:,1])
plt.title("Customer Clusters using UMAP")
plt.show()
```

Output

